Assignment 3: Workflows with Airflow

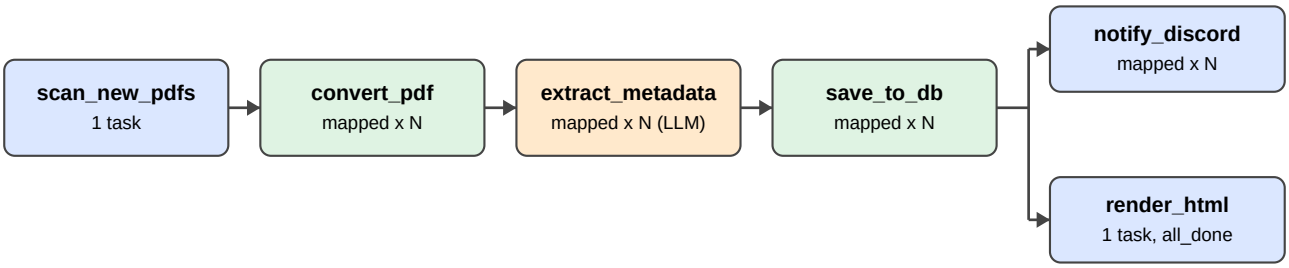
INFO-H420 Process Modeling, Data Pipelines, and Responsible AI

Team: [Name 1], [Name 2], [Name 3] (Group [N])
Code repository: https://github.com/[your-username]/[your-repo]

1. Goal and approach

We built one scheduled Airflow DAG, `paper_pipeline`, that turns research PDFs dropped into a Google Drive-synced `papers/` folder into a searchable study library. For each new PDF the DAG converts it to Markdown, asks a locally served LLM for a title, summary, keywords and category as JSON, stores the result in a local SQLite database, regenerates a static `index.html`, and sends a Discord notification. Airflow runs locally with `airflow standalone`; the SQLite file stays on the local machine, outside the synced folder.

2. DAG design and task dependencies



We used **dynamic task mapping**: `scan_new_pdfs` returns the list of new filenames and the next tasks are expanded once per PDF. A paper that fails (for example a scanned PDF or an LLM error) does not block the others, and because it is never written to the database it is picked up again on the next run. `extract_metadata` is limited to one running instance (`max_active_tis_per_dag=1`) so the local LLM is not overloaded, and `render_html` uses `trigger_rule="all_done"` so the page is rebuilt even when nothing was new. The DAG runs every 10 minutes with `catchup=False` and `max_active_runs=1` to avoid processing a file twice.

3. Implementation

Step	Implementation
Find new PDFs	List `papers/*.pdf`, subtract filenames already in SQLite (filename is the unique key).
PDF to Markdown	PyMuPDF4LLM `to_markdown()`. The .md file is written locally and only its path is passed via XCom (keeps XComs small).
Local LLM	Python `openai` client pointed at Ollama (`localhost:11434/v1`) or LM Studio. JSON mode (`response_format=json_object`) is used, with a fallback to a plain request if the server rejects it. Input is truncated to about 12,000 characters. The category is chosen from a fixed list in the prompt.
JSON handling	`json.loads` (tolerating a Markdown code fence), then a light check that title/summary/category are non-empty strings and keywords is a list of strings. On failure the call is **retried once**; after the second failure the task fails (and Airflow retries the task).
SQLite	Built-in `sqlite3`, table `papers` (filename UNIQUE, title, summary, keywords as JSON string, category, processed_at). Upsert on filename.
Static HTML	Single self-contained `index.html` with the data embedded as JSON, a search box and a category dropdown (plain JavaScript, text inserted via `textContent`). Written to a temp file then renamed so Drive never syncs a half-written page.

Step	Implementation
Discord	One webhook POST per new paper (title, category, keywords, short summary). The URL comes from an environment variable and is never committed.

All logic lives in `dags/pipeline_utils.py` and the DAG file only wires it into tasks, so the logic (JSON retry, validation, new-file detection, DB upsert, HTML generation) is covered by unit tests that run without Airflow, an LLM or network access.

4. Evidence that the workflow runs

The pipeline was run locally with Airflow 3.4.0 (`airflow standalone`), Ollama 0.40.15 serving `llama3.2:latest` (3.2B parameters, Q4_K_M) and PyMuPDF4LLM. The raw model metadata, database rows and run states are in `docs/evidence/` of the repository.

Run / check	Observed result
Duplicate run (nothing new)	First tests used a local demo copy of the library folder. Run succeeded. `scan_new_pdfs` found no new file, so convert, extract, save and notify were **skipped**; `render_html` still succeeded. The database kept its 1 known record.
New file added	A second PDF was converted, described by the LLM, saved and rendered in about 65 s (LLM step about 54 s). The database went from 1 to 2 records.
Run on the Drive library	Three further PDFs, kept in the Google Drive-synced `ResearchLibrary/papers` folder (`https://drive.google.com/drive/u/0/folders/1w574K7Nj-hlWkswR0RV8XLOhHTexfpDn`), were processed in one run (4 min 18 s). All six tasks succeeded and the three per-paper tasks were mapped x3 (Figure 1). The database then held 5 records, and `output/index.html` was written back into the synced folder.
JSON from the model	All 5 papers returned valid JSON with title, summary, keywords and category; no task failed. The records are in `sqlite_records_evidence.json`.
Discord	Three `notify_discord` tasks (one per paper) completed successfully (Figure 2). Delivery in the channel is shown in Figure 3.
Search and filter	The generated page filtered by search "Autoencoder" and category "Machine Learning" returned exactly one paper (Figure 4).
Time zones	Timestamps in the database and run metadata are UTC; the Airflow UI shows Europe/Paris (UTC+1).

Example records produced by the pipeline

File	Title (LLM)	Category
2410.05889.pdf	Deep Learning-Based Fault Identification in Condition Monitoring	Machine Learning
2101.11539.pdf	Autoencoder-based Condition Monitoring and Anomaly Detection Method for Rotating Machines	Machine Learning
A-generic-framework-...pdf	A generic framework for scenario exercises using models applied to water-resource management	Data Engineering
A-GIS-tool-...pdf	A GIS tool for modeling anthropogenic noise propagation in natural ecosystems	Data Engineering
A-dynamic-model-...pdf	A dynamic model for optimally phasing in CO2 capture and storage infrastructure	Data Engineering

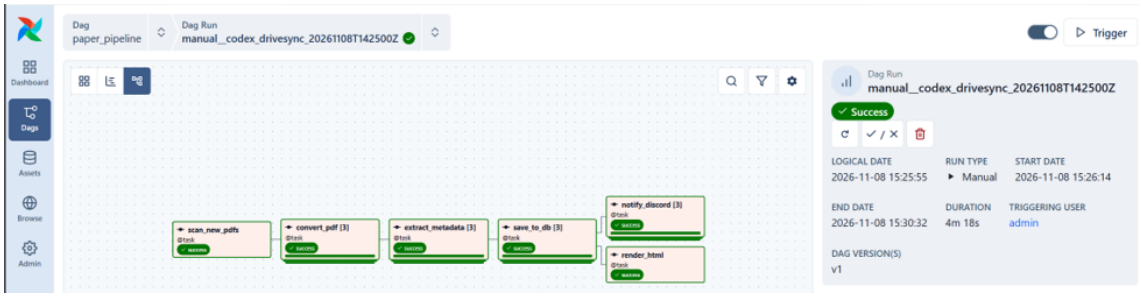


Figure 1. Airflow graph view of the run on the Drive library: all six tasks succeeded; convert_pdf, extract_metadata and save_to_db were mapped over 3 papers (run duration 4 min 18 s).

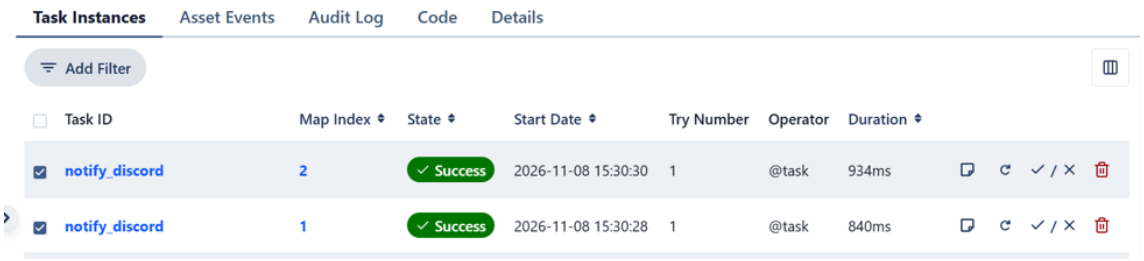


Figure 2. notify_discord task instances (one per paper, state Success).

[INSERT SCREENSHOT: Figure 3. Discord channel showing the received notifications. (save as docs/shot_discord.png)]

[INSERT SCREENSHOT: Figure 4. Generated index.html filtered with search "Autoencoder" and category "Machine Learning". (save as docs/shot_html.png)]

[INSERT SCREENSHOT: Figure 5. Google Drive folder (ResearchLibrary) with papers/ and output/index.html. (save as docs/shot_drive.png)]

5. Limitations

Only the start of each paper is sent to the LLM, so summaries of long papers reflect the introduction and early sections. The category list in the prompt is generic: the three environmental-modelling papers were all labelled "Data Engineering" because no better option existed, which shows how model output depends on the prompt and on the 3B model's size. One summary contains a literal `<sup>` tag copied from the PDF text; the page escapes it, so it appears as text. Scanned PDFs without a text layer are not supported (no OCR), and filenames are assumed to be unique. Seven unit tests (`tests/test_utils.py`) cover JSON validation and retry, new-file detection, the database upsert, HTML escaping and the missing-webhook case; the LLM and Discord calls are tested only through the live runs above.

6. Reproducing

Install steps, configuration and run instructions are in the README of the repository: https://github.com/[your-username]/[your-repo]